

Contents

Product Backlog	1
Epic 1 — Bewegung & Überleben	2
Epic 2 — Angriff	3
Epic 3 — Strategie & Zustände	5
Epic 4 — Qualität (optional)	6
Epic 5 — Turniervorbereitung	7

Product Backlog

Gemeinsames Backlog für alle drei Teams. Jede Story hat Akzeptanzkriterien und Story Points (Planning-Poker-Skala 1/2/3/5 — 1 = trivial, 5 = anspruchsvoll für diese Gruppe). Reihenfolge der Epics ist eine Empfehlung, keine Pflicht-Sortierung.

Für jede Story gibt es eine Musterlösung für den Dozenten in [dozent/loesungen.md](#) — bitte nicht an Schüler weitergeben, bevor sie es selbst versucht haben.

Referenz-API (siehe auch `framework/arena/Models.kt`):

```
interface RobotBrain {  
    val name: String  
    fun decide(sensors: Sensors): Action  
}  
  
// sensors.self: eigener RobotState (position, health, alive)  
// sensors.others: alle anderen lebenden Roboter  
// sensors.arenaWidth / arenaHeight: Rastergröße (10x10)  
// Action: Move(Direction) | Shoot(Direction) | Wait  
// Direction: NORTH, EAST, SOUTH, WEST
```

Epic 1 — Bewegung & Überleben

Story 1.1 — Zufällige Bewegung (Warmup)

Story Points: 1

Als Team möchte ich, dass mein Bot sich bei jedem Tick in eine zufällige Richtung bewegt, damit ich sehe, dass mein Bot überhaupt korrekt ins Spiel eingebunden ist.

Akzeptanzkriterien:

- `decide()` gibt bei jedem Aufruf `Action.Move(direction)` mit einer zufällig gewählten `Direction` zurück.
- Der Bot ist in der Bot-Auswahl der App sichtbar und bewegt sich sichtbar über die Arena.

Story 1.2 — Am Rand bleiben / Zentrum meiden

Story Points: 2

Als Team möchte ich, dass mein Bot sich eher am Rand der Arena aufhält (schwerer zu treffen, wenn weniger Gegner in Reihe/Spalte kommen), damit er nicht sofort im Zentrum kampiert und leicht von mehreren Seiten getroffen wird.

Akzeptanzkriterien:

- Der Bot berücksichtigt `sensors.self.position` relativ zu `sensors.arenaWidth/arenaHeight`.
- Befindet sich der Bot zu nah am Zentrum, bewegt er sich in Richtung eines Randes.

Story 1.3 — Flucht bei niedriger Gesundheit

Story Points: 3

Als Team möchte ich, dass mein Bot flieht, wenn seine Gesundheit unter 20 fällt, damit er nicht sinnlos weiterkämpft und vielleicht überlebt.

Akzeptanzkriterien: - Ist `sensors.self.health < 20`, bewegt sich der Bot vom nächsten Gegner weg (nicht auf ihn zu). - Ist `sensors.self.health >= 20`, verhält sich der Bot wie zuvor (normale Logik, z.B. aus Epic 2). - Gibt es keinen Gegner mehr (`sensors.others.isEmpty()`), wirft der Bot keine Exception (z.B. `Action.Wait` als Fallback).

Epic 2 — Angriff

Story 2.1 — Dauerfeuer in feste Richtung

Story Points: 1

Als Team möchte ich, dass mein Bot ständig in eine feste Richtung schießt, damit ich die grundlegende Schuss-Mechanik ausprobieren kann.

Akzeptanzkriterien: - `decide()` gibt immer `Action.Shoot(direction)` mit einer festen `Direction` zurück. - Im Testduell gegen `StillstandBot` (aus `bots/examples`) wird sichtbar Schaden verursacht, wenn die Ausrichtung passt.

Story 2.2 — Auf nächsten Gegner in Sichtlinie zielen

Story Points: 3

Als Team möchte ich, dass mein Bot gezielt auf einen Gegner schießt, der sich exakt in seiner Reihe oder Spalte befindet, damit Treffer nicht dem Zufall überlassen sind.

Akzeptanzkriterien: - Der Bot prüft für alle `sensors.others`, ob einer davon gleiches `x` (dann NORTH/SOUTH) oder gleiches `y` (dann EAST/WEST) wie `sensors.self.position` hat. - Gibt es mehrere Kandidaten, wird der nächstgelegene gewählt (kleinster Abstand). - Steht kein Gegner in Sichtlinie, tut der Bot etwas anderes (z.B. bewegen, siehe Story 2.3) statt sinnlos ins Leere zu schießen.

Story 2.3 — Gegner verfolgen, wenn nicht in Schusslinie

Story Points: 3

Als Team möchte ich, dass mein Bot einem Gegner hinterherläuft, wenn er ihn nicht direkt anvisieren kann, damit er aktiv Kämpfe sucht statt nur zu warten.

Akzeptanzkriterien: - Steht kein Gegner in Sichtlinie (siehe 2.2), bewegt sich der Bot einen Schritt in Richtung des nächstgelegenen Gegners (kleinste Distanz). - Kombiniert mit Story 2.2: steht ein Gegner in Sichtlinie -> schießen, sonst -> verfolgen.

Epic 3 — Strategie & Zustände

Story 3.1 — Einfache Zustandsmaschine (Patrouille / Angriff / Flucht)

Story Points: 5

Als Team möchte ich, dass mein Bot zwischen mehreren klar benannten Verhaltenszuständen wechselt, damit sein Verhalten nachvollziehbar und erweiterbar ist.

Akzeptanzkriterien: - Mindestens 3 Zustände sind erkennbar (z.B. PATROUILLE wenn kein Gegner in der Nähe, ANGRIFF wenn ein Gegner erreichbar ist, FLUCHT wenn `health < 20`). - Der Zustand wird bei jedem `decide()`-Aufruf neu aus den aktuellen `sensors` bestimmt (kein gespeicherter Zustand nötig, aber erlaubt — z.B. über eine `var` im Bot). - Jeder Zustand führt zu klar unterscheidbarem, im Log sichtbarem Verhalten.

Story 3.2 — Zielpriorisierung bei mehreren Gegnern

Story Points: 3

Als Team möchte ich, dass mein Bot bei mehreren möglichen Zielen sinnvoll auswählt (z.B. den schwächsten oder nächsten Gegner), damit er nicht wahllos das erste beste Ziel angreift.

Akzeptanzkriterien: - Bei mehreren Gegnern in `sensors.others` wählt der Bot nach einem klaren Kriterium (z.B. niedrigste `health` zuerst, oder kleinste Distanz). - Das Kriterium ist im Code klar benannt/kommentiert.

Story 3.3 — Kür-Aufgabe (freie Wahl)

Story Points: 5

Als Team möchte ich eine eigene Idee umsetzen, die über die vorgegebenen Storys hinausgeht, damit ich meine eigene Kreativität einbringen kann.

Akzeptanzkriterien: - Team formuliert die Story selbst (kurz, 1-2 Sätze) und trägt sie als eigene Karte ins Board ein. - Dozent bestätigt kurz, dass die Idee mit der bestehenden RobotBrain-API umsetzbar ist, bevor das Team startet. - Beispiele: Bewegungsmuster, das Gegner in eine Ecke drängt; Bot, der sich an der Wand entlang bewegt; einfache "Ich schieße nur, wenn ich sicher treffe"-Heuristik.

Epic 4 — Qualität (optional)

Story 4.1 — Unit-Test für eigene Entscheidungslogik

Story Points: 2

Als Team möchte ich einen einfachen Unit-Test für einen Teil meiner `decide()`-Logik schreiben, damit ich das in der Praxis ausprobiert habe.

Akzeptanzkriterien: - Mindestens ein Test in `src/test/kotlin/bots/teamX/...` (Ordner ggf. neu anlegen) ruft `decide()` mit einem selbst gebauten `Sensors`-Objekt auf und prüft das Ergebnis mit `assertEquals/assertTrue`. - Test ist grün (`./gradlew test`).

Story 4.2 — Testduell protokollieren

Story Points: 1

Als Team möchte ich ein Testduell gegen einen Beispiel-Bot durchführen und das Ergebnis kurz notieren, damit wir den Fortschritt unseres Bots dokumentieren.

Akzeptanzkriterien: - Testduell gegen mindestens einen Bot aus `bots/examples` durchgeführt. - Ergebnis (Sieg/Niederlage/Unentschieden, ungefährender Verlauf) in 1-2 Sätzen notiert (z.B. auf der Board-Karte oder in einer Notiz-Datei im Team-Ordner).

Epic 5 — Turniervorbereitung

Story 5.1 — Finale Bot-Version festlegen

Story Points: 1

Als Team möchte ich am Ende von Tag 3 eine klar benannte, funktionierende Bot-Version für das Turnier bereithaben, damit die Integration beim Dozenten reibungslos läuft.

Akzeptanzkriterien: - `teamXBots`-Liste in `TeamXBots.kt` enthält genau die Bot-Instanz(en), die im Turnier antreten sollen. - Projekt kompiliert fehlerfrei (`./gradlew build`).

Story 5.2 — Strategie-Kurzvorstellung

Story Points: 1

Als Team möchte ich unsere Bot-Strategie in 1-2 Sätzen vorstellen können, damit die anderen Teams und der Dozent beim Review verstehen, was unser Bot tut.

Akzeptanzkriterien: - Team kann in eigenen Worten erklären, wann ihr Bot angreift, flieht oder patrouilliert.